



Tecnológico de Monterrey

Evidencia 1 - Actividad Integral de Conceptos Básicos y Algoritmos Fundamentales (Evidencia Competencia)

Estructura de Datos

Elena María Barrios Jordan

|

A01771338

20 Sep 2026

Reflexión

Con esta Evidencia 1 desarrollé un programa de C++ para poder leer los registros de archivos de texto distintos ya incluidos, primero ordenarlos por fecha y hora, después buscar registros dentro de un rango de fechas y guardar ese resultado en un archivo de salida.

Durante el desarrollo implementé 6 algoritmos de ordenamiento: Insertion Sort, Quick Sort, Merge Sort, Bubble Sort, Selection Sort y Swap Sort.

El objetivo principal de esta actividad para mí fue entender que un algoritmo no solamente debe funcionar correctamente, sino que la parte que incluí de medir el tiempo me parece sumamente importante para poder hacer los análisis y comparaciones en tiempo y recursos necesarios. Al probar los diferentes algoritmos con dos archivos distintos, pude observar que el orden inicial de los datos puede afectar mucho el tiempo de ejecución de algunos algoritmos dependiendo su complejidad y lo desarrollaré más adelante.

Usamos en el programa también dos búsquedas binarias para encontrar registros dentro del rango de fecha y hora propuestos por el usuario. Sobre esta parte fue muy importante resolver la problemática de los casos que existen de registros que tienen exactamente el mismo timestamp.

Como platicaba en el archivo de evidencias, cada prueba me ayudó a entender el funcionamiento individual de cada algoritmo y entender porque cada complejidad tiene diferentes resultados, me gustó entenderlo porque antes solo lo había memorizado.

Me pude dar cuenta que Insertion Sort fue diferente dependiendo del archivo. Por ejemplo en log607-1.txt, que tiene los registros super desordenados, tardó 3.426 seg, en cambio, en log607-2.txt que está casi ordenado, tardó solo 0.009 seg. Aquí es cuando podemos ver que Insertion Sort puede ser más su mejor caso de $O(n)$ cuando los datos están ya casi ordenados.

Hablando de Bubble Sort fue casi igual ya que en el primer archivo tardó 3.752 seg, mientras que en el segundo tardó 0.007 seg lo que es un mundo de diferencia. Entonces concluye que dependiendo del nivel de complejidad algorítmica es cuando debemos ver si conviene o no ponerle cierto tipos de datos más ordenados o menos ordenados.

Quick Sort por otro lado tuvo diferentes resultados, en log607-1.txt tardó 0.018 seg, pero en log607-2.txt tardó 1.849 seg. Es importante decir que usamos el último elemento como pivote como vimos en clase, por lo que el orden inicial de los datos pueden afectar mucho el comportamiento.

Merge Sort fue más constante por su complejidad de $O(n \log n)$ en el mejor y peor caso, tardó 0.023 seg en el primer File y 0.023 seg en el segundo File. Aquí vemos que el orden inicial de los Logs no cambia casi nada los resultados obtenidos por Merge Sort.

Selection Sort tardó 0.195 seg en el primero y 0.124 seg en el segundo. Su comportamiento se podría decir que fue constante o estable, por lo que su mejor y peor caso son $O(n^2)$.

Al final Swap Sort tardó aproximadamente 3.784 seg en el primero y 0.126 seg en el segundo. como tiene dos ciclos for anidados decimos que en su mejor y peor caso son $O(n^2)$.

Los resultados me ayudaron a entender que la complejidad de la teoría nos sirve para analizar el comportamiento general de un algoritmo, pero no significa que dos algoritmos con una misma complejidad siempre tarden los mismos seg.

Por ejemplo, Selection Sort y Swap Sort tienen comportamiento cuadrático, pero sus tiempos medidos no fueron exactamente iguales.

Para el parámetro de decisión en predicción utilicé un margen de error de 20% aunque creo que puedo aumentarlo un poco más ya que después de hacer múltiples pruebas con cada algoritmo me di cuenta que intentar hacer una predicción es muy difícil, para tenerla lo más acertada posible podríamos considerar el tamaño de los datos, el orden inicial del archivo y la complejidad del algoritmo.

Otra parte importante del proyecto fue implementar las búsquedas binarias. La primera encuentra la primera posición la que su timestamp es mayor o igual al inicio del rango. La segunda encuentra la primera posición en que el timestamp es mayor que el final del rango. Lo hice así para que la búsqueda pudiera incluir todos los registros que tenían el mismo timestamp aunque estuvieran repetidos.

USO DE LA IA

Durante el proyecto utilicé inteligencia artificial como apoyo para comprender conceptos, revisar código y detectar/explicar errores, en los casos de errores que investigar por mi cuenta solo generaban más errores le pedía que me corrigiera el código y posteriormente que lo explicara además muchas veces yo lo cambié porque no me gustaba su corrección. No utilicé las respuestas de la IA simplemente copiándolas sin revisarlas. En varias ocasiones tuve que comparar las explicaciones con mi propio código o mis ejemplos de clase los cuales me ayudaron muchísimo para el desarrollo de éste proyecto.

Algunos de los prompts que utilicé para la IA fueron:

“Revisa este código de ordenamiento ‘Swap Sort’ y dime si las complejidades de mejor y peor caso que estoy utilizando son correctas.” “Explicame que es lo que crea esas diferencias”

“Dime una función simple de usar con la que pueda medir el tiempo de los algoritmos de ordenamiento que estoy usando”

La IA también cometió algunos errores durante el desarrollo.

Uno de los errores más importantes fue con Swap Sort. Ya que al inicio me había dicho que podía llegar a un mejor caso de $O(n)$. Cuando yo revisé el código me di cuenta que el algoritmo tiene dos ciclos for que se ejecutan aunque los datos ya estén ordenados. Entonces la complejidad correcta es $O(n^2)$ tanto en el mejor como en el peor caso. Esta revisión me enseñó que debo confiar en mi criterio y cuestionar cuantas veces sienta necesario a la IA.

Algo de lo más importante que aprendí de este proyecto fue que no existe un algoritmo que tenga exactamente el mismo comportamiento para todos los tipos de datos.

También aprendí que las estructuras de datos y las búsquedas deben diseñarse pensando en cada caso específico y poner mucha atención de que tipo son los datos en dónde es que lo vamos a usar.

Aprendí a utilizar la inteligencia artificial como una herramienta de apoyo y no como sustituto de mi cerebro o revisión personal. Las respuestas de la IA me ayudaron a avanzar, entender y aprender pero también encontré explicaciones que necesitaban ser corregidas así que para que mi trabajo pudiera desarrollarse de la mejor forma tuve que usar mi criterio personal, mis aprendizajes de clase y horas de estudio propio.

Al terminar esta actividad me doy cuenta que aprendí herramientas que se pueden utilizar en un sinnúmero de utilidades en el mundo real y me siento preparada para seguir aprendiendo y abriendo mi mente a formas de solucionar problemas de forma eficiente y ordenada.